

PA2 - Write-up

Student / ID: Okassova Islamiya / 242322

Link: islamyiaok-cpu/PA1_VScode

Wedge, $T = 8$ s, cube axis (1,1,1), horizontal orbit, camera 0 (eye (0, 2.5, 7), FOV 45°); near = 1, far = 15.

E1 - Order of transformations

$M = R_{\text{orbit}} * T * R_{\text{self}} * S$. Each glMatrix call right-multiplies ($M \leftarrow M * X$), so the call written last acts on the vertex first: scale $\rightarrow$ self-spin $\rightarrow$ move to radius 2.5 $\rightarrow$ orbit.

(a) Swap orbit rotation and translation:

Prediction: both rotations now act at the origin, before T. Both are about y, so they add up to one spin of $2\pi/8 + 2 = 2.785$ rad/s, and T then moves the solid once. The wedge should stay at the fixed point (2.5, 0, 0) and only spin in place; no orbit.

Result: Figures 1 / 2: the wedge stays at the same place beside the cube and only its orientation changes.

Explanation: R_{orbit} is applied before T, so it turns the wedge around its own centre, not around the cube.

(b) Swap translation and self-spin:

Prediction: T moves the wedge to $x = 2.5$ first, so R_{self} now rotates it about the origin (the cube). Self-spin becomes orbital motion at 2.785 rad/s (period $2\pi/2.785 = 2.26$ s), and the wedge always shows the same face to the cube.

Result: Figures 3/4: the wedge circles the cube much faster than $T = 8$ s, keeping the same face towards it.

Explanation: R_{self} is applied after T, so it acts on the offset position, and its angle adds to the orbit angle.

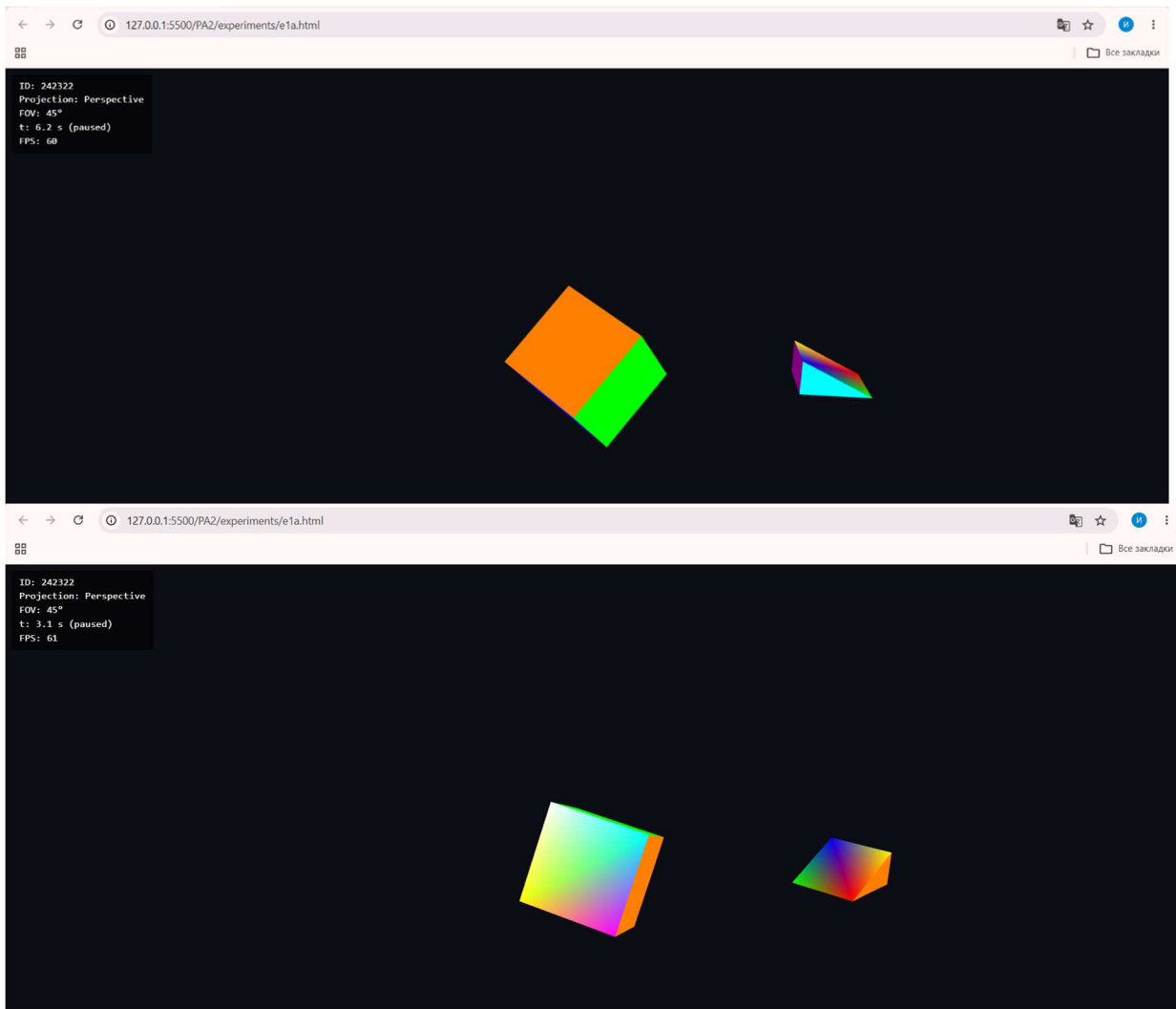


Fig. 1 - 2: T and R_{orbit} swapped.

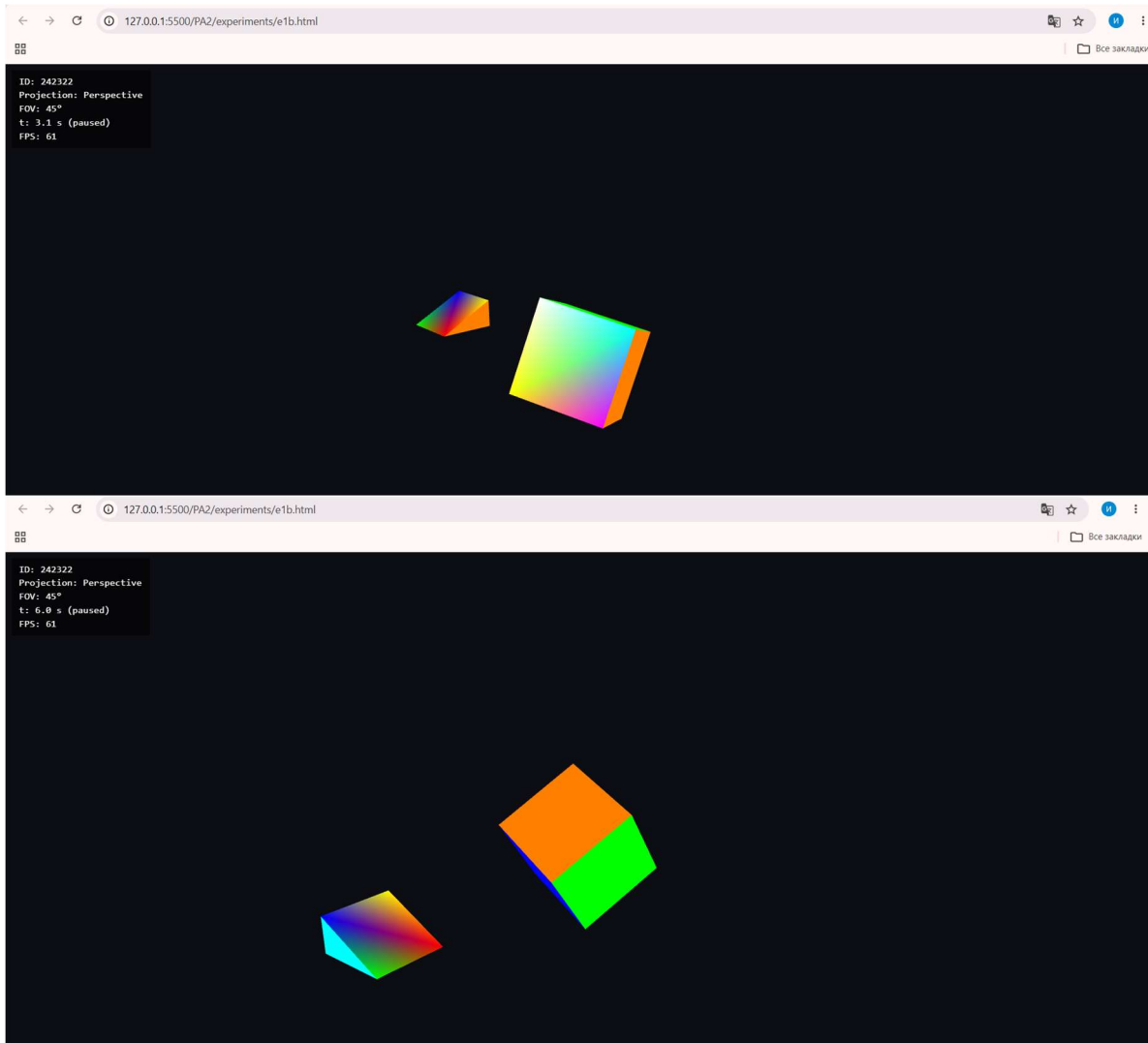


Fig. 3 - 4: T and R_{self} swapped.

E2 - Three statements in Chapter 4

- (a) The comment says "(X)" but $[0, 1, 0]$ is the y-axis (x is $[1, 0, 0]$): the non-zero component of the axis vector selects the axis, so the cube rotates about z and y.
- (b) Removing geometry outside the view volume is clipping: after the vertex shader, primitives are clipped against $-w \leq x, y, z \leq w$, crossing triangles are cut. Face culling is a separate optional stage that drops back-facing triangles by winding order.
- (c) w is the homogeneous coordinate, not a quaternion component: $w = 1$ is a point, $w = 0$ a direction, so translation moves only points. In perspective $w_{clip} = -z_{view}$ and the divide by w gives foreshortening. Quaternions are a separate rotation representation.

Prediction: the translation $(2, 3, 4)$ shifts the point ($w = 1$) to $(3, 3, 4)$ but leaves the direction ($w = 0$) unchanged.

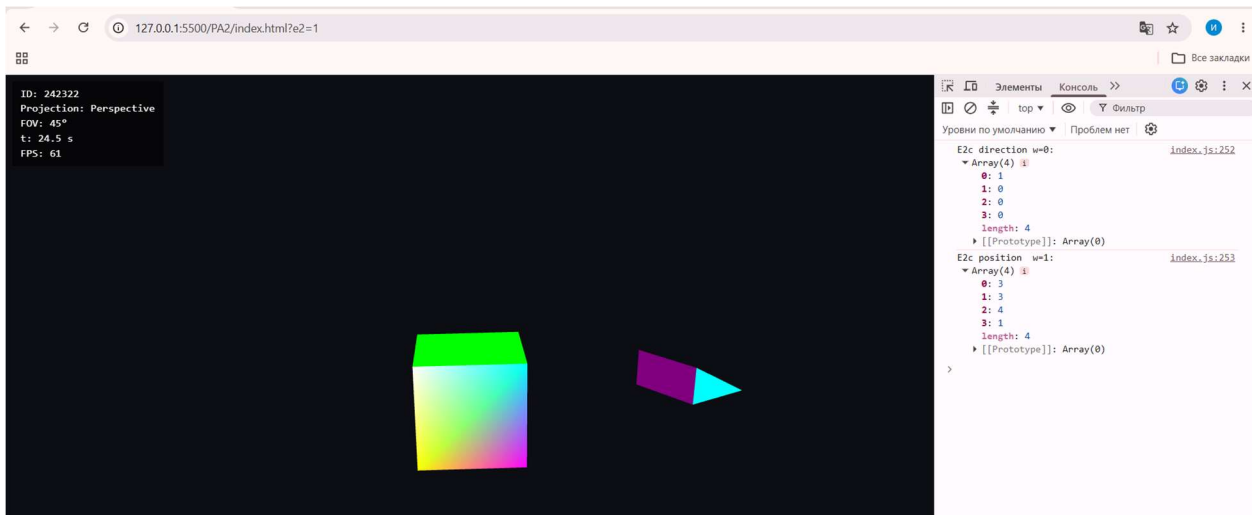


Fig. 5 ($E2c$, $?e2=1$)

E3 - Perspective vs orthographic

Pose $?t=0$ (paused). Measured on the line cube ($?wire=1$): front-right (nearest) and back-right (farthest) vertical edges.

Prediction: ratio ≈ 1.14 in perspective (depth ratio 7.90 / 6.96), 1.00 in orthographic.

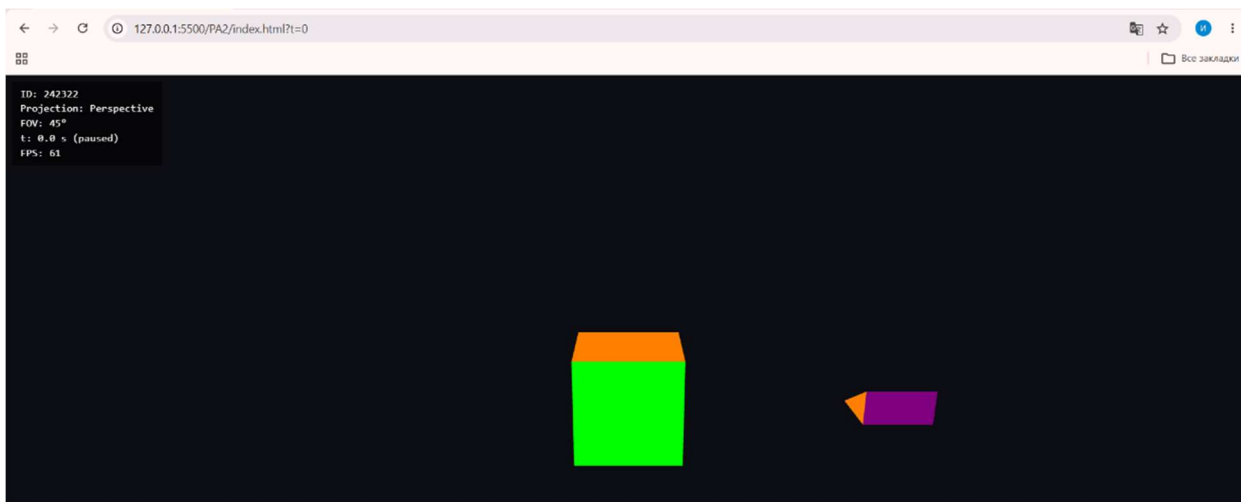


Fig. 6: Perspective projection



Fig. 7: Orthographic projection

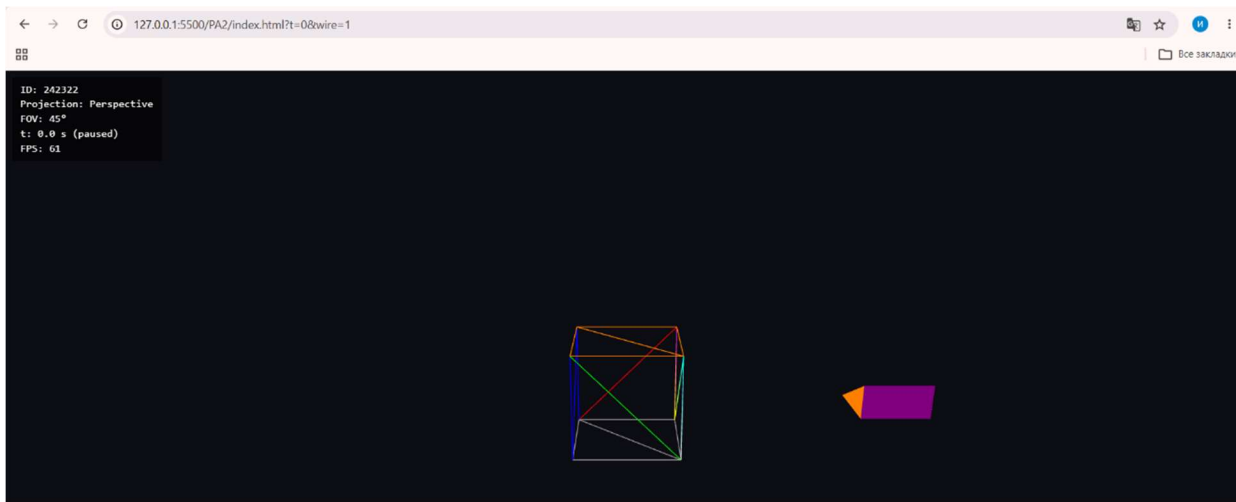


Fig. 8: Wireframe cube, perspective

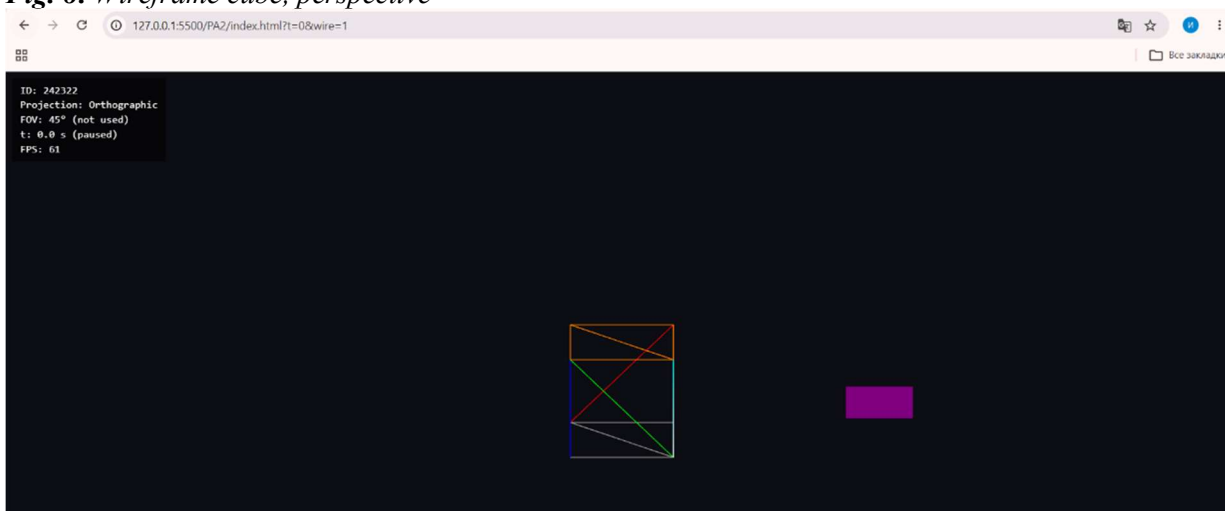


Fig. 9: Wireframe cube, orthographic

Result: Measured on the two line-cube screenshots above.

Projection	Edge	Top (x, y)	Bottom (x, y)	Length px	Near / far
Perspective	near	(1048, 541)	(1044, 700)	159.05	1.13
Perspective	far	(1037, 497)	(1033, 638)	141.06	
Orthographic	near	(1039, 544)	(1039, 694)	150.00	0.99
Orthographic	far	(1039, 490)	(1039, 641)	151.00	

Explanation. Perspective stores view depth in w ($w_{clip} = -z_{view}$) and the divide by w scales size by $1/w$. Near edge depth ≈ 6.96 , far ≈ 7.90 , so the near edge is longer by $\approx 7.90/6.96 = 1.14$. The orthographic matrix keeps $w = 1$, nothing is divided and parallel edges stay equal; the 1 px difference is reading error.

PA1 offset trick: $x += o_x(z + 0.5)$, $y += o_y(z + 0.5)$ shifts points in proportion to z without rescaling and keeps parallels parallel, so it resembles the orthographic projection.

E4 - Near, far and aspect

The orbit radius is 2.5, so the eye-solid-centre distance lies in $[\sqrt{4.5^2 + 2.5^2}, \sqrt{9.5^2 + 2.5^2}] = [5.15, 9.82]$. Wedge vertices are at most $0.75 \times 0.8 = 0.6$ from its centre, so the closest vertex is ≥ 4.55 and the farthest ≤ 10.42 (cube ≤ 8.30). Depth along the view axis is slightly smaller than distance, but at the closest approach it is still about $5.08 - 0.6 \approx 4.5$. near = 1 is well below this and not tiny; far = 15 exceeds 10.42 with margin.

(a) Prediction: with near = 6 the plane starts cutting the wedge when its centre is at view depth ≈ 6 ($t \approx 4.83$) and removes it completely when every vertex is closer than 6 (around $t = 6$).

Result: ?t=4.8&near=6 (left) shows the wedge sliced; ?t=6&near=6 (right) shows only the cube.

Explanation: at $t \approx 4.83$ the wedge centre is at view depth $7.43 - 0.9417 \times 1.52 = 6.0$, so the near plane cuts through it. At $t = 6$ the whole wedge (depth ≈ 4.5 -5.7) is in front of the plane and is clipped away.

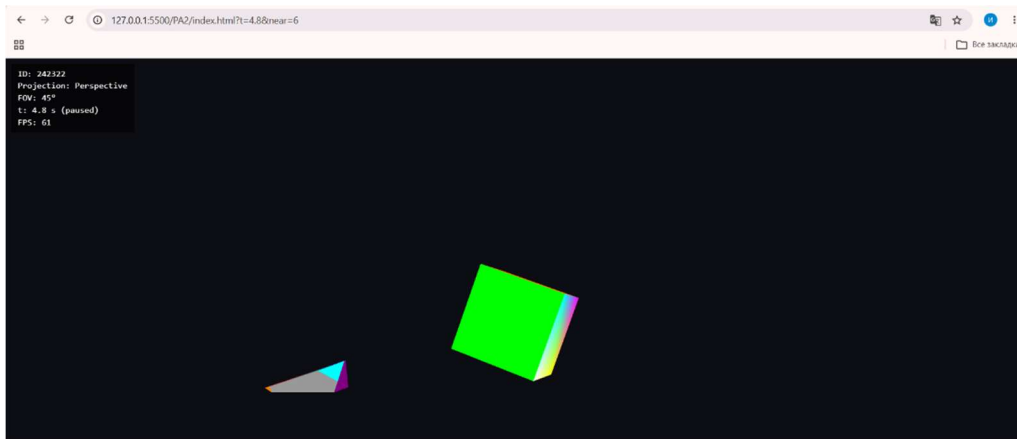


Fig. 10: the near plane slices the wedge.

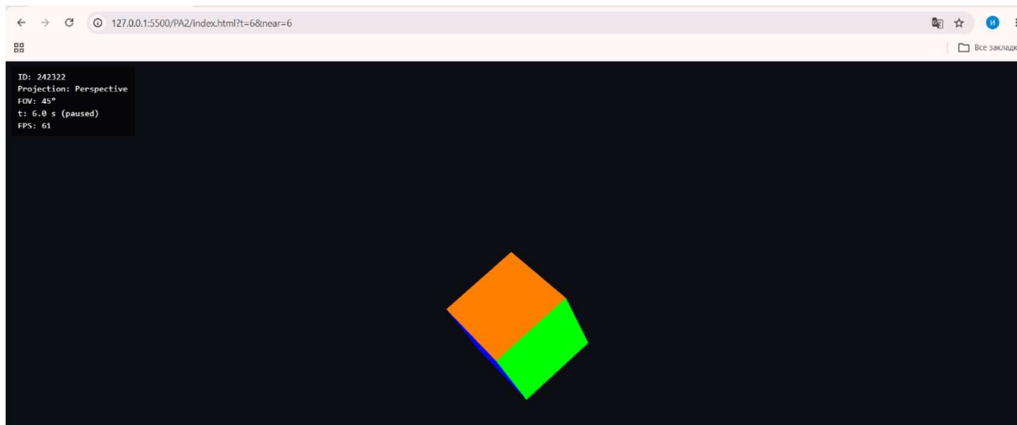


Fig. 11: the wedge is clipped, the cube remains

(b) Prediction: with aspect fixed to 1 the x-scale becomes $1/\tan(\text{fov}/2)$ instead of $1/(\text{aspect} \times \tan(\text{fov}/2))$, so in a wide window everything is stretched horizontally by width/height.

Result: ?aspect=1 in a wide window: the cube becomes a wide box.

Explanation: the projection no longer compensates for the non-square viewport.

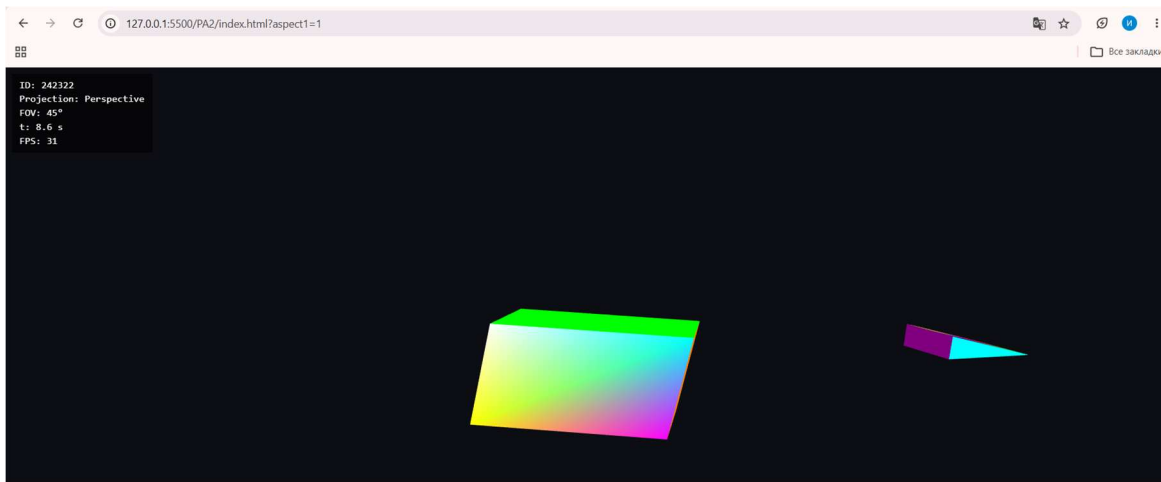


Fig. 12: the cube is stretched horizontally.

E5 - Frame-rate independence

Prediction: dt equals the real frame time ($\sim 1/60$ s, with a maximum slightly higher but far below the 0.1 s clamp). On a slower CPU dt grows, but $t += dt$ still accumulates real seconds, so one orbit still takes $T = 8$ s. Without the clamp, a hidden tab produces one huge dt and the scene jumps.

dt log (?logdt=1, 5 s, no throttling)

Result: $n = 301$, mean = 0.01661 s, max = 0.01780 s; FPS 61. **Explanation:** the mean dt equals $1/\text{fps}$, so dt is the real frame time. The maximum is far below 0.1 s, so the clamp is inactive in normal running.

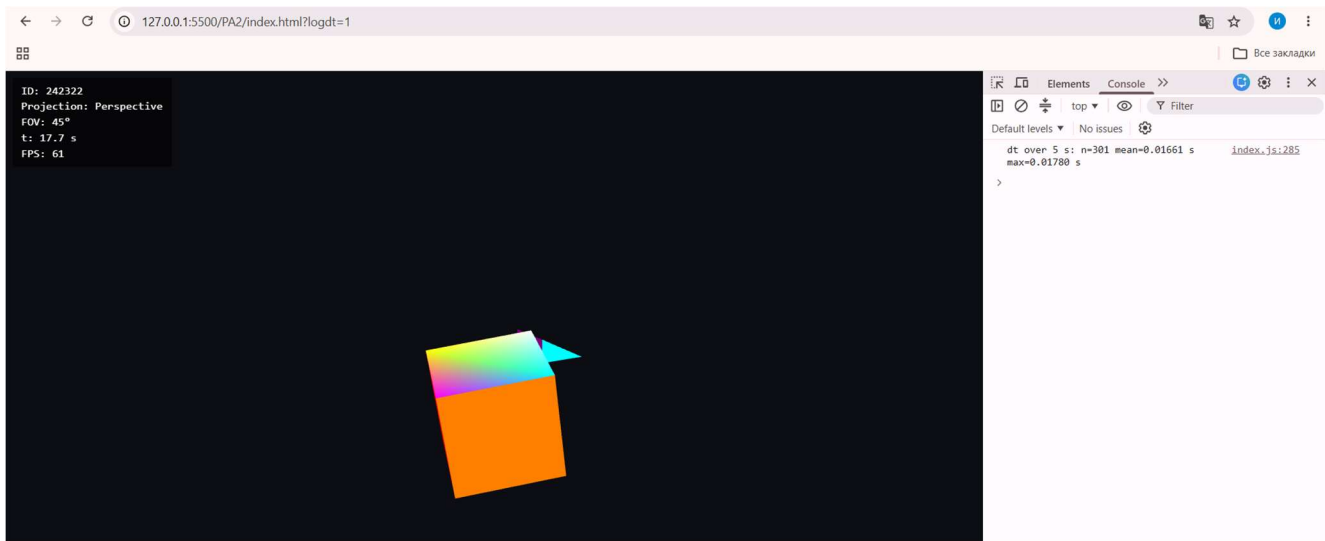


Fig. 13

6 x CPU throttling (?logorbit=1)

Result: the status label shows FPS ~ 31 ; the console logs orbit 1: simulated $t = 8.02x$ s at real time = 8.02x s, and orbit 2: simulated $t = 16.2x$ s at real time = 16.2x s.

Explanation: t accumulates real time, so a lower frame rate only makes the steps larger. One orbit still takes $T = 8$ s (the extra 0.0x s is the detection frame).

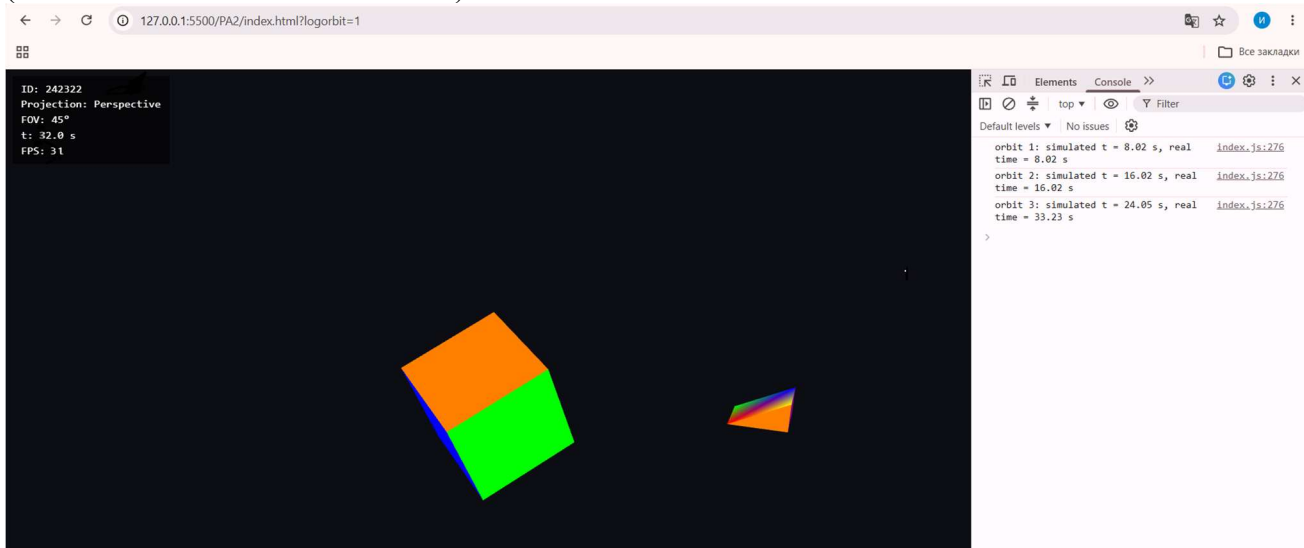


Fig. 14

No clamp (?noclamp=1).

Result: before leaving, $t = 3.7$ s. After the tab was hidden for about 12.8 s, the console shows large dt : 12.83 s.

Explanation: hidden tabs stop `requestAnimationFrame`, so the first frame sees the whole absence and t jumps by 12.83 s, i.e. 1.60 orbit. With the clamp the jump is at most $0.1/8 \times 360^\circ = 4.5^\circ$.

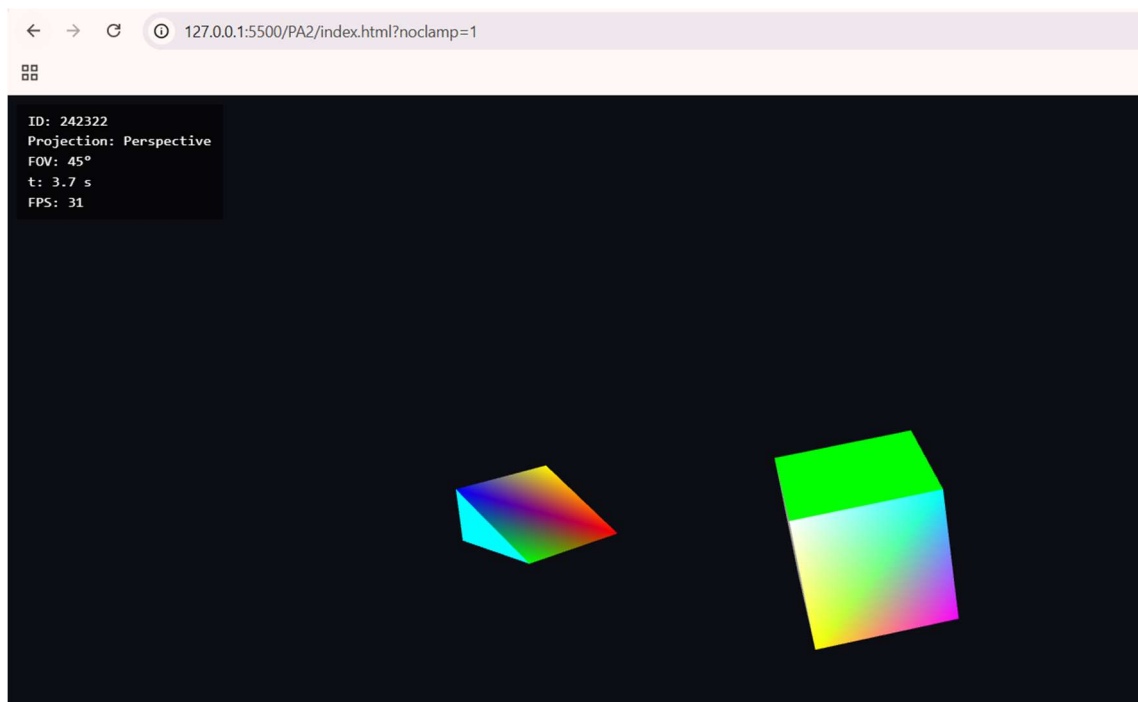


Fig. 15 Before

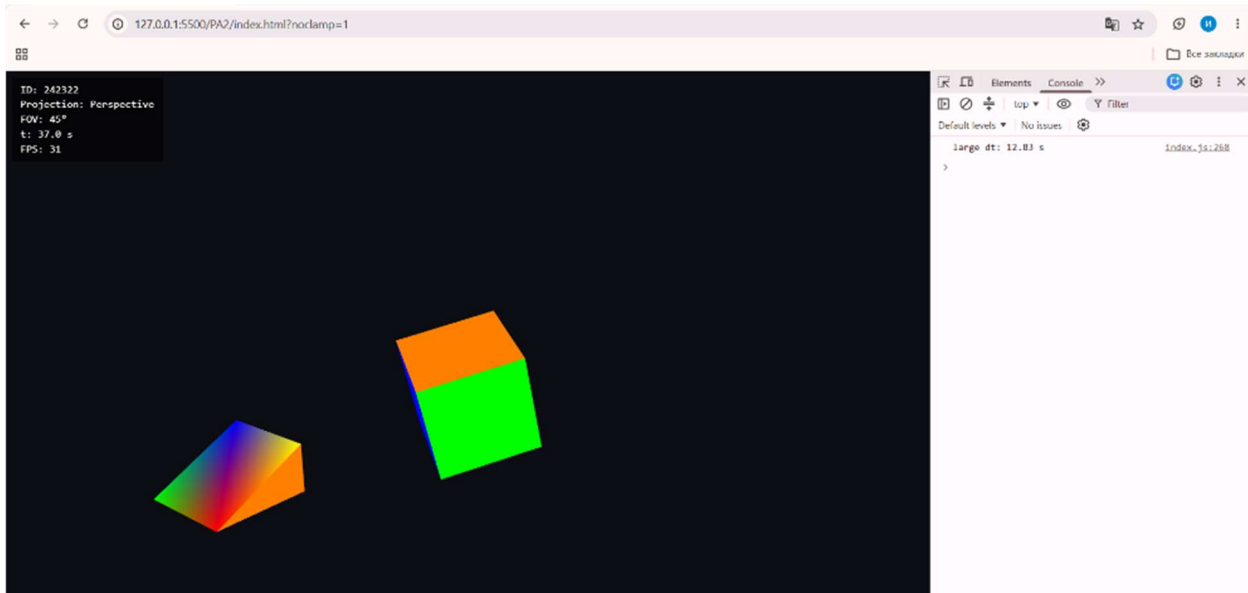


Fig. 16 *after*

E6 - One vertex by hand

?t=1.7453: spin $1.2 * 1.7453 = 2.0944$ rad = 120° about (1,1,1).

Canvas 1135 x 791, aspect = 1.43489, FOV 45° , near 1, far 15.

Vertex $v = (0.5, 0.5, -0.5, 1)$.

1. Model. 120° about (1,1,1) cycles the axes, $(x, y, z) \rightarrow (z, x, y)$: $M*v = (-0.5, 0.5, 0.5, 1)$ (code: (-0.5, 0.50002, 0.49998, 1); t is rounded).

2. View.

$z_cam = (0, 2.5, 7)/7.43303 = (0, 0.33634, 0.94174)$;

$x_cam = up * z_cam = (1, 0, 0)$;

$y_cam = z_cam * x_cam = (0, 0.94174, -0.33634)$.

$p_rel = M*v - eye = (-0.5, -1.99998, -6.50002)$.

$V*M*v = (x_cam*p_rel, y_cam*p_rel, z_cam*p_rel) = (-0.50000, 0.30273, -6.79401)$.

3. Projection.

$\tan 22.5^\circ = 0.41421$:

$P00 = 1/(1.43489 * 0.41421) = 1.68250$,

$P11 = 2.41421$, $P22 = 16/(1 - 15) = -1.14286$,

$P23 = 30/(1 - 15) = -2.14286$, $w = -z_view$.

$Clip = (-0.84125, 0.73085, 1.14286 * 6.79401 - 2.14286, 6.79401) = (-0.84125, 0.73085, 5.62172, 6.79401)$.

4. NDC = clip.xyz / w = $(-0.12382, 0.10757, 0.82745)$, inside $[-1, 1]^3$: visible.

Console (vec4.transformMat4) agrees to 5 decimals

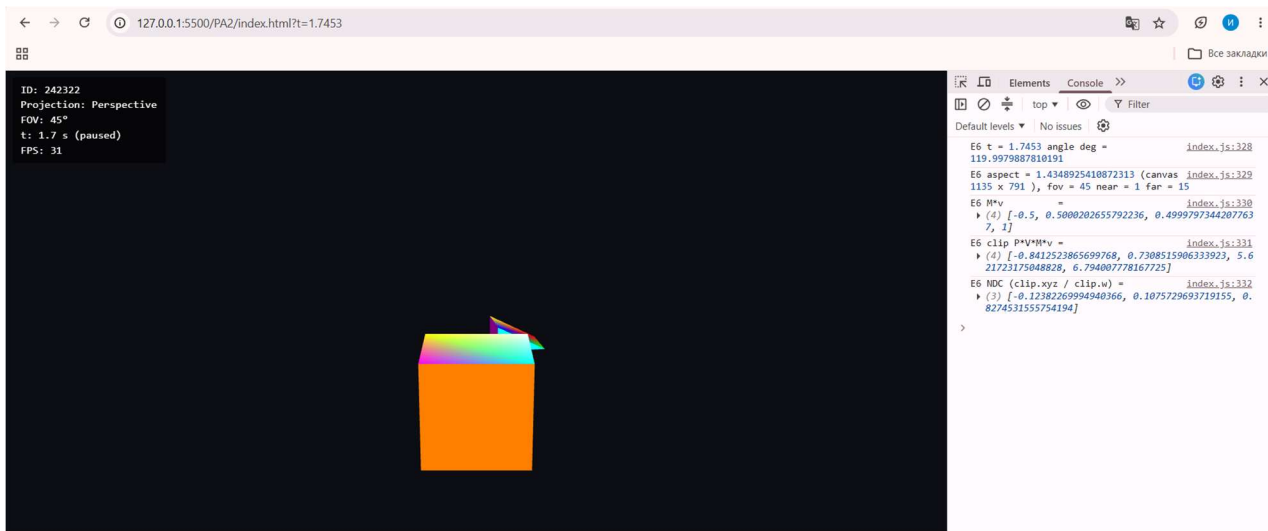


Fig. 17

Development log

1. Bug: blank page, "Cannot read properties of undefined (reading 'create')".
2. Found with the console and `typeof mat4` → 'object': glMatrix 2.8.1 defines `mat4` as a global, not `glMatrix.mat4`.
3. Fixed in commit `ed286c2`.